Relazione 2

Allora per questa Relazione ho prima avviato il container wn1 perché contiene tutti I packetti necessari.

```
CONTAINER ID   IMAGE                 COMMAND                  CREATED       STATUS          P
ORTS                                        NAMES
b14c8250800c   retilab/ubuntu-hpc    "/usr/sbin/sshd -D"   4 days ago    Up 24 minutes   0
.0.0.0:2202->22/tcp, [::]:2202->22/tcp   wn2
d73a249025a1   retilab/ubuntu-hpc    "/usr/sbin/sshd -D"   4 days ago    Up 24 minutes   0
.0.0.0:2201->22/tcp, [::]:2201->22/tcp   wn1
```

Nella cartella performance ci sono sutto cartelle qui vediamo il tree delle cartelle. Performace rispetto a : Timing , matrixMul ,network storage(se ci sono file csv è perche avevo gia fato prima di scrivere la relazione).

```
root@wn1:~/2-Performance/Performance# tree
.
|-- matrixMul
|   |-- CPU
|   |   |-- mm
|   |   |-- mm.c
|   |   |-- mm.sh
|   |   |-- mm.slurm
|   |   |-- mm.wn1.csv
|   |   |-- omp_mm
|   |   |-- omp_mm.c
|   |   |-- omp_mm.wn1.csv
|   |   `-- omp_mm_scaling.slurm
|   |-- GPU
|   |   |-- matrixMul
|   |   `-- matrixMul.slurm
|   `-- PLOTTING
|       |-- matrixMul.png
|       |-- matrixMul.py
|       |-- mm.wn1.csv
|       `-- omp_mm.wn1.csv
|-- network
|   |-- band.py
|   |-- band_lat.sh
|   |-- band_lat.slurm
|   |-- lat.py
|   |-- mpi_bandwidth.c
|   `-- mpi_latency.c
|-- storage
|   |-- disktest.sh
|   `-- disktest.slurm
`-- timing
    |-- timing
    |-- timing.c
    |-- timing.csv
    |-- timing.py
    |-- timing.sh
    `-- timing.slurm

8 directories, 29 files
root@wn1:~/2-Performance/Performance#
```

Directory Timing

Allora nella cartella timing ho esegiuito il file timing.sh con il commando “sh timing.sh” che compila il file timing.c e crea timing.csv. Ecco il detaglio del file timing.c.

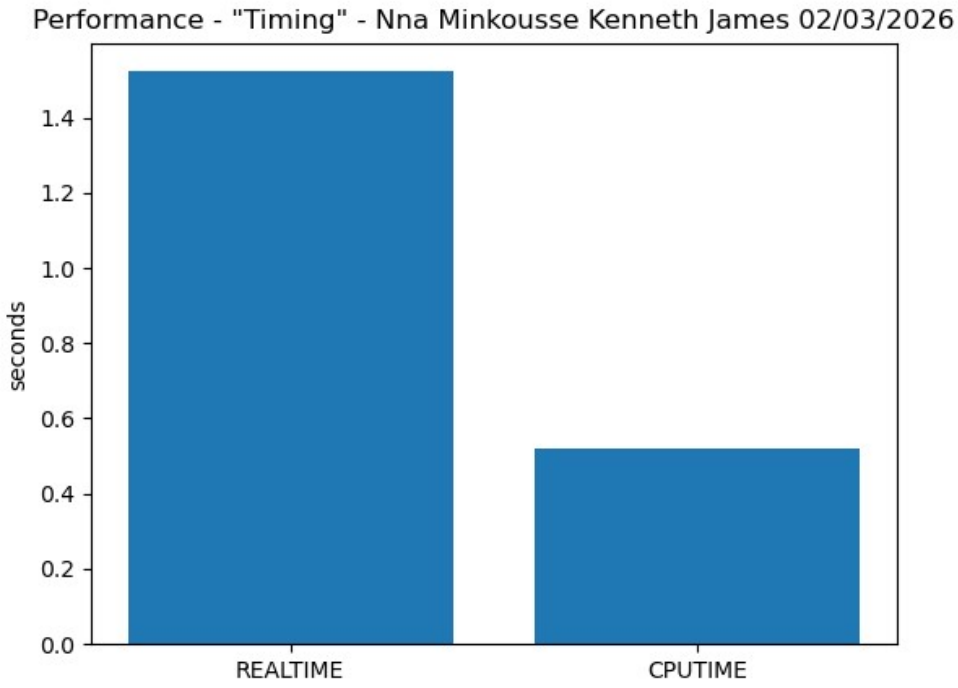
```
int main( int argc, char **argv )
  {
    struct timespec start_realtime, stop_realtime, start_cputime, stop_cputime;
    double rt, ct;

   clock_gettime( CLOCK_REALTIME ,          &start_realtime) ;
   clock_gettime( CLOCK_PROCESS_CPUTIME_ID , &start_cputime) ;

   sleep (1);      //  increment real time

   int i;
   double j;
   for (i = 0; i < 500000000; i++) { j+=i*0.001; }   // increment real time  e  CP
   printf( "# j=%.4lf \n", j);

   clock_gettime( CLOCK_REALTIME           , &stop_realtime ) ;
   clock_gettime( CLOCK_PROCESS_CPUTIME_ID , &stop_cputime ) ;

    rt    = (double) ( stop_realtime.tv_sec - start_realtime.tv_sec )
            + (double) ( stop_realtime.tv_nsec - start_realtime.tv_nsec )
            / BILLION;

   ct     = (double) ( stop_cputime.tv_sec - start_cputime.tv_sec )
            + (double) ( stop_cputime.tv_nsec - start_cputime.tv_nsec )
            / BILLION;

    printf( "   timer          time\n");
    printf( "   REALTIME      %.4lf \n", rt);
    printf( "   CPUTIME       %.4lf \n", ct);
```

Questo programa al inizio inizia 2 clock cioa il wall clock e CPU clock e a la fine lo stoppano. Fa la print a la fine del 2 tempi.

Ecco il CSV ottenuto:

```
# j=124999999750000.0000
  timer          time
  REALTIME      1.5227
  CPUTIME       0.5215
```

Ecco addesso il file timing.png generato da timing.py



Matrixmul

Directory CPU

Allora per questo matrix multiplication dal codice abbiamo 2 matrice quadrate con dimezione(1024 x 1024).
il codice mm.c avvia e stoppa il REAL_TIME_CLOCK dopo la multipicazione di 2 matice (con e sensa la libreria gsl). Senza la libreria lo fa in maniera naive. Alla fine con un file csv si vede il tempo impiegato.
Ecco il file csv ottenuto:

(Con mm.c => Naive) → usando il compilatore gcc e gcc+gsl(libreria)

```
# N  gflop  tempo(s)  GFLOP/s  Hostname   Comment
1024, 2.15,  1.05,   2.05,  wn1, gcc-wn1
# N  gflop  tempo(s)  GFLOP/s  Hostname   Comment
1024, 2.15,  0.63,   3.39,  wn1, gcc+gsl-wn1
```

Si vede che usado solo il compilatore gcc il tempo è di 1.05sec e 2.05 GFLOP/s rispetto a 0.63sec e 3.39 GFLOP/s usando gcc e la libreria gsl . (quindi con l'aiuto di gsl aumenta le performance).

Con openMP(sul omp_mm.c):

Addesso facciamo la versione naive parallelizzata con openMP.
Cioè compilo il file omp_mm.c e eseguo il file eseguibile(omp_mm).
Ecco il resultato:

```
root@wn1:~/2-Performance/Performance/matrixMul/CPU# gcc -fopenmp omp_mm.c -o omp
_mm
root@wn1:~/2-Performance/Performance/matrixMul/CPU# ./omp_mm
#N  gflop  time(s)  GFLOPS   Hostname   Comment nthreads
 1024,  2.15,  0.77,   2.81, wn1, no-comment,  12
root@wn1:~/2-Performance/Performance/matrixMul/CPU#
```

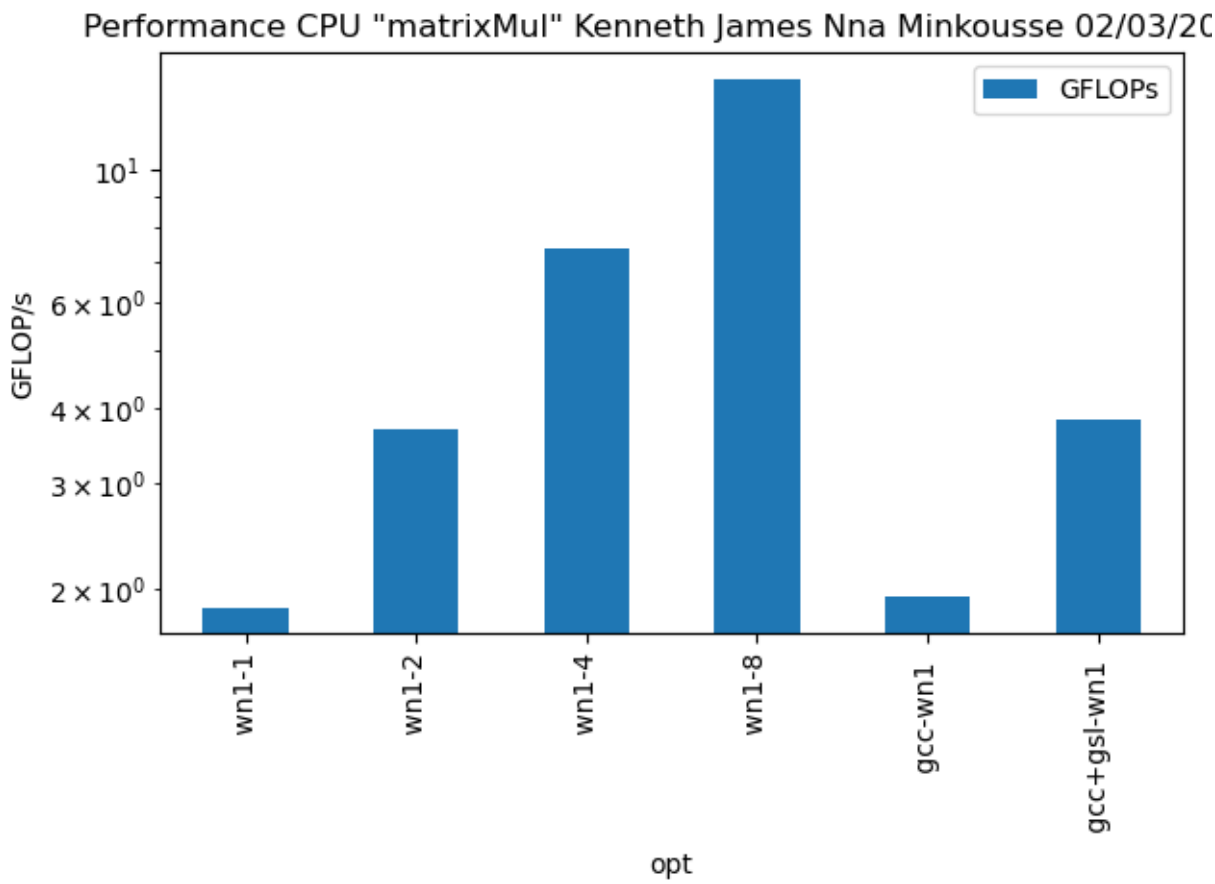
Si vede dal output che abbiamo usato 12 threads e e il tempo è di 0.77s che e maggiormente buono rispetto a 1.05s usando il metodo naive soppra.

Directory PLOTTING

Dopo aver copiato tutti I file *.csv nella directory PLOTTING ecco il resultato dopo aver compilato il file python:

```
root@wn1:~/2-Performance/Performance/matrixMul/PLOTTING# cp ../CPU/*csv .
root@wn1:~/2-Performance/Performance/matrixMul/PLOTTING# ls
matrixMul.py  mm.wn1.csv  omp_mm.wn1.csv
root@wn1:~/2-Performance/Performance/matrixMul/PLOTTING# python3 matrixMul.py
      N  gflop  tempo  GFLOPs Hostname            opt   nt
0  1024   2.15   1.15    1.87      wn1          wn1-1  1.0
1  1024   2.15   0.58    3.71      wn1          wn1-2  2.0
2  1024   2.15   0.29    7.40      wn1          wn1-4  4.0
3  1024   2.15   0.15   14.12      wn1          wn1-8  8.0
4  1024   2.15   1.10    1.95      wn1       gcc-wn1   NaN
5  1024   2.15   0.56    3.83      wn1   gcc+gsl-wn1   NaN
root@wn1:~/2-Performance/Performance/matrixMul/PLOTTING#
```

Di seguito ecco il .png generato:



Conclusione:

- Possiamo concludere che per quanto riguarda l'immagine .png al crescere dei numeri di threads aumenta il GFLOP/s.

- Usando la libreria gsl con il compilatore gcc, aumenta il GFLOP/s rispetto a quando si usa solo il compilatore gcc.

Band-latency

Bandwidth(B): This is simply the amount of data transmitted per second in a communication channel between to process. Measured in MB/s and it dependence on the amount of data(message passed).
Benchmark → bandwidth.c

Latency(L): This is the time needed to transmit data from process A to process B.

Benchmark → latency.c

If the communication of this processes are on the same node the have a shared memory and the Operating system takes the control but if the are on different nodes the processes will communicate via a network and thus the will be more latency.

When I start the script (sh band_lat.sh)

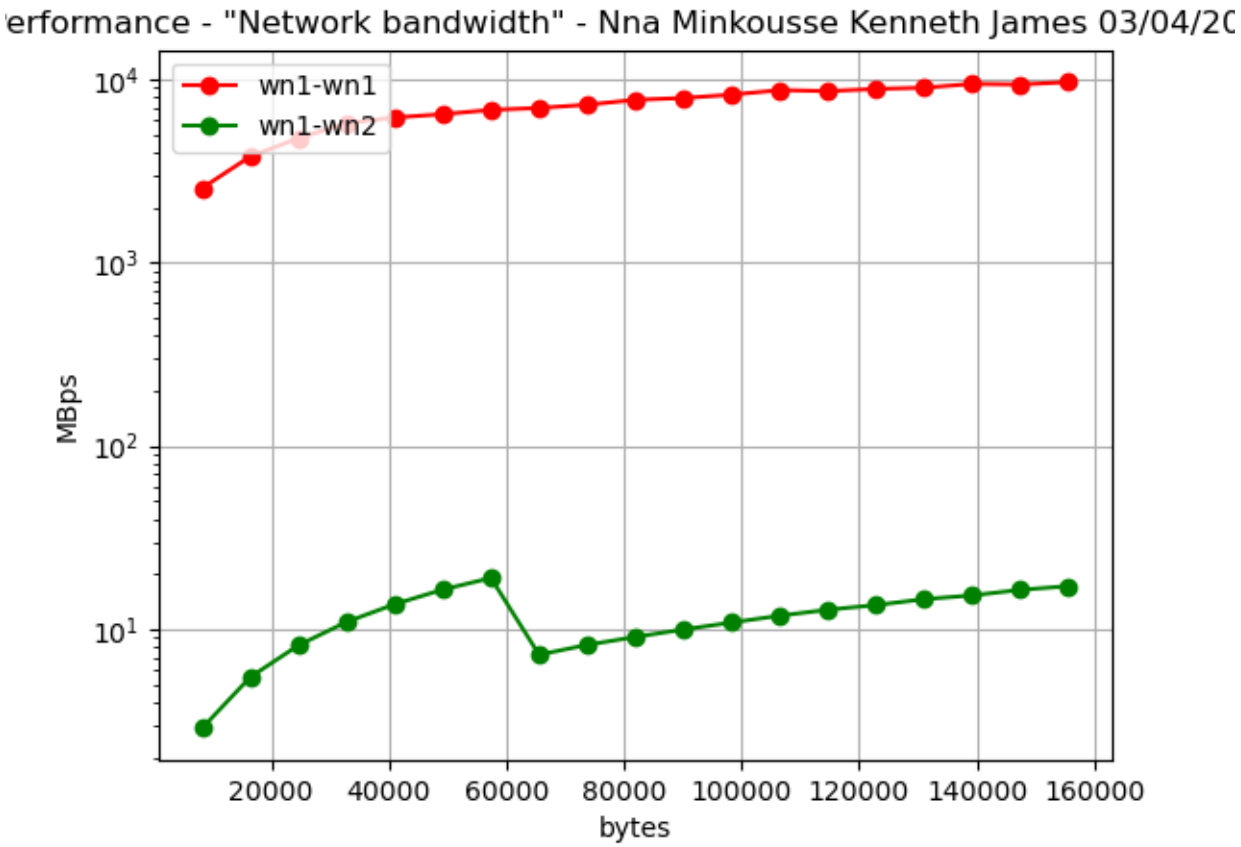
Result Bandwidth test:

```
# ****************** MPI/POE Bandwidth Test *****************
Message start size= 8192 bytes
Message finish size= 163480 bytes
Incremented by 8192 bytes per iteration
Roundtrips per iteration= 100
MPI_Wtick resolution = 1.000000e-03
************************************************************
task    0 is on wn1 partner=   1
task    1 is on wn1 partner=   0
************************************************************
Message size:     8192 *** best  /  avg  / worst (MB/sec)
   task pair:    0 -    1:    2763.37 / 2548.07 / 111.79
   OVERALL AVERAGES:          2763.37 / 2548.07 / 111.79

Message size:    16384 *** best  /  avg  / worst (MB/sec)
   task pair:    0 -    1:    4267.78 / 3784.59 / 373.88
   OVERALL AVERAGES:          4267.78 / 3784.59 / 373.88
```

Result Latency test:

```
task 0 has started...
Beginning latency timing test. Number of reps = 100.
*****************************************************
Rep#       T1                T2              deltaT
   1  0.00000000  0.00001293  0.00001293
   2  0.00001874  0.00001992  0.00000119
   3  0.00002197  0.00002283  0.00000086
   4  0.00002489  0.00002583  0.00000094
   5  0.00002771  0.00002855  0.00000083
   6  0.00003045  0.00003126  0.00000081
   7  0.00003319  0.00003402  0.00000083
   8  0.00003595  0.00003683  0.00000088
   9  0.00003884  0.00003967  0.00000083
  10  0.00004171  0.00004267  0.00000096
  11  0.00004478  0.00004579  0.00000101
  12  0.00004776  0.00004854  0.00000077
  13  0.00005047  0.00005129  0.00000082
```

Result on the graph:



Storage Directory

Disktest:

when I execute the file disktest.sh I obtain the following result

```
root@wn1:~/2-Performance/Performance/storage# sh disktest.sh
Write to file:
2048+0 records in
2048+0 records out
2147483648 bytes (2.1 GB, 2.0 GiB) copied, 7.87096 s, 273 MB/s

Read from file:
2048+0 records in
2048+0 records out
2147483648 bytes (2.1 GB, 2.0 GiB) copied, 3.5809 s, 600 MB/s
root@wn1:~/2-Performance/Performance/storage#
```

From this result we can see that when writing 2GB data to a file it takes 7.87096 sec and has a bandwidth of 273 MB/s

When Reading the same data from a file it takes 3.5809 sec and a 600MB/s bandwidth .

In conclusion with a high bandwidth we have less time to perform the action.